

DESIGN

1. Overall Approach

EnergyPass is designed around a single constraint: it must work as a fully self-contained out-of-tree LLVM plugin with no dependencies on LLVM analysis passes from the FunctionAnalysisManager (FAM). On macOS, loading an out-of-tree plugin that also registers or queries FAM analyses causes duplicate-symbol crashes at plugin load time because the plugin's analysis registry collides with the host opt binary's registry. The entire design — loop detection, model loading, remarks emission — is structured to avoid any FAM query while still producing correct and useful output.

Within that constraint the design is organized into five independent components that compose linearly: instruction classification, loop depth analysis, energy model loading, per-block/per-function accumulation, and output emission. Each component is a pure function or a stateless struct with no shared mutable state between function invocations, making the pass safe for concurrent use in multi-threaded opt pipelines.

2. Component Design

- *Instruction Classification:* A single `classify(Instruction&)` function maps every LLVM IR instruction to one of seven `InstrCategory` values using LLVM RTTI priority order: Memory (`LoadInst`, `StoreInst`, `AtomicRMWInst`, `AtomicCmpXchgInst`) is checked first because memory instructions can have integer result types and would otherwise be misclassified; Branch (`BranchInst`, `SwitchInst`, `IndirectBrInst`, `ReturnInst`) and Call (`CallInst`, `InvokeInst`) follow; then type-based dispatch — `VectorType` with floating-point elements → `FloatVector`, scalar floating-point → `FloatScalar`, integer or pointer type → `Integer`, everything else → `Other`. This ordering ensures that `LoadInst` with an i64 result is always Memory, not Integer — a correctness requirement for energy accounting. The `Other` category acts as a catch-all for metadata instructions, phi nodes, `alloca`,

getelementptr, and bitcast; these have near-zero energy cost (0.1) and their presence in Other is also a signal of structural anomaly when they appear in unexpectedly large proportions.

- *Loop Depth Analysis (computeLoopDepths)*: Loop nesting depth is computed with a two-phase custom algorithm that requires no FAM analyses. Phase one (computeDominators) runs an iterative data-flow fixed-point over the CFG to build the dominator sets for every basic block — the standard textbook algorithm initializing the entry block's dominator set to {entry} and all other blocks to the full block set, then intersecting predecessor dominator sets until convergence. Phase two identifies loop headers as successors of blocks that they dominate (back-edges), then counts how many loop headers dominate each block to get its nesting depth. Depth is converted to a frequency weight by $\text{depthToWeight}(d) = 10^d$. This means straight-line code has weight 1, a single-loop body has weight 10 (assuming ~10 iterations), a doubly-nested body has weight 100, and so on. The heuristic deliberately uses a round number: it is designed to produce correct relative rankings between hot and cold code, not to predict absolute cycle counts.
- *Energy Model (EnergyModel+loadModel)*: The EnergyModel struct holds a string-keyed cost map loaded either from data/energy_model.json (via LLVM's llvm::json parser) or from hard-coded defaults. The JSON has a top-level 'architectures' object with 'aarch64' and 'x86_64' sub-objects, each mapping category name strings to normalized float costs. Architecture is auto-detected from the module's target triple via llvm::Triple. If the JSON file is absent, unreadable, or missing the required arch key, the pass falls back to built-in defaults and prints a warning to stderr — it never fails silently and never aborts the compilation. The AArch64 defaults (Integer=1.0, FloatScalar=1.5, FloatVector=2.5, Memory=2.0, Branch=1.5, Call=3.0, Other=0.1) are validated against three published sources: Tsoutsouras et al. IEEE TCAD 2013, ARM Cortex-A55 Software Optimization Guide DDI0618, and Retkowski & Stechele DATE 2017. All seven categories pass the published range check (see evaluation_results.md).
- *Accumulation (BlockResult / FunctionResult)*: analyseBlock() iterates every instruction in a BasicBlock, calls classify(), accumulates category counts and energy into a BlockResult struct, and captures the first DebugLoc it finds for source-location annotation. The outer

FunctionResult loop in EnergyPass::run() aggregates BlockResults into per-function totals, keeping both static (unweighted) and weighted energy so the amplification factor (weighted ÷ static) can be computed for validation.

- *Output (stderr report + LLVM remarks)*: Two output channels run unconditionally and conditionally. The stderr tabular report always fires: it prints per-block and per-function category breakdowns with counts, static energy, weighted energy, and percentage share. The LLVM remarks channel fires only when opt is invoked with -pass-remarks-analysis=energy. Remarks are emitted via LLVMContext::diagnose(OptimizationRemarkAnalysis), which bypasses the FAM entirely while still hooking into opt's -pass-remarks-output YAML pipeline. Function-level remarks fire for every non-declaration function; block-level remarks fire only for hot blocks (loop depth > 0) that have a DebugLoc, the subset most actionable for a repair loop.

3. Alternatives Considered and Rejected

- *FAM LoopAnalysis*: The natural LLVM API for loop depth. Rejected because querying analyses through FAM from an out-of-tree plugin registration causes duplicate-symbol crashes on macOS Apple Silicon at -O1. The custom dominator walk produces identical results for well-structured CFGs and avoids the crash entirely.
- *OptimizationRemarkEmitter (ORE) via FAM*: The standard LLVM remarks API. Also requires FAM registration, triggering the same crash. Replaced with direct LLVMContext::diagnose() calls, which are documented to be the correct mechanism for passes that do not own an ORE.
- *Profile-guided energy measurement*: Running instrumented binaries to count actual instruction executions would give far more accurate energy estimates. Rejected for portability: it requires a target machine, a runtime, and a second compilation pass. The static model runs on any host with LLVM installed.
- *MLIR / FIR passes*: MLIR offers richer type information and a more expressive dialect system that would allow finer-grained energy characterization (e.g., distinguishing affine

loop nests from general loops). Rejected for this project because it adds a heavy dependency chain (MLIR build, separate CMake configuration) and moves the analysis further from the LLVM IR level required by the assignment.

- *Flat instruction counting without a cost model:* Simply counting instructions gives a proxy for code size but not energy. Rejected because it treats a load and an integer add identically, which is a $2\times$ error at minimum and up to $50\times$ error for cache-miss loads. The JSON cost model adds negligible overhead ($\sim$ microseconds per function) while dramatically improving accuracy.
- *Per-instruction power tables from hardware PMU:* Using hardware performance monitoring unit data (e.g., perf on Linux, Instruments on macOS) to measure per-instruction power. Rejected as out-of-scope: it requires privileged hardware access, vendor-specific tooling, and a calibration run on physical hardware. The static model is explicitly designed to avoid this dependency.

4. Design Tradeoffs

- *Accuracy vs. portability:* The static model trades dynamic accuracy (branch prediction, cache behaviour, DVFS) for zero hardware dependency and full portability. Every test in the evaluation suite passes the literature range check, confirming the model is accurate enough for relative energy ranking even without runtime data.
- *Simplicity vs. precision:* The 10^{depth} heuristic deliberately uses a round number rather than attempting to estimate loop trip counts statically. Static trip-count estimation requires alias analysis and induction-variable analysis (both FAM-dependent), introduces significant error for input-dependent loops, and adds substantial code complexity. The heuristic produces correct relative rankings across all five loop-containing test functions.
- *Self-contained vs. extensible:* Making the pass FAM-free means it cannot reuse results from other LLVM analyses (alias analysis, scalar evolution, etc.) that would allow more precise memory cost modelling. This is the deliberate cost of the portability constraint. The JSON model is the extension point: teams that want more precise costs for a specific microarchitecture can update `data/energy_model.json` without touching the pass source.